‘fancyvrb’ 宏包
LaTeX 中的花式 verbatim

Timothy Van Zandt
普林斯顿大学
美国普林斯顿

封装与文档编写：

Denis Girou（法国 CNRS/IDRIS），

Sebastian Rahtz†（英国 Elsevier）

以及

Herbert Voß（德国柏林自由大学）

2026 年 11 月 9 日

摘要

本宏包提供了非常完善的机制，用于读取和排版 verbatim TeX 代码。用户可以完成诸如更换字体族与字号、给行编号、给代码示例加边框、给文本着色以及按条件处理文本等常见任务。

问题报告请发送至 hvoss@tug.org

致谢：Claudio Beccari, Karl Berry, Jean-François Burnol, Philippe Esperet, Michael Friendly, Mario Hassler, Ohad Kammar, Mikhail Kolodin, Oliver Kopp, Andreas Matthias, Frank Mittelbach, Rolf Niepraschk, Will Robertson, Ulrich M. Schwarz, Thomas Siegel, Clemens Steinke, Vladimir Volovich, 以及 Yukai Chou

目录

I 宏包 fancyvrb 4

1 简介 4

2 脚注中的 verbatim 内容 4

3 增强的 verbatim 命令 5

4 Verbatim 环境 5

4.1 verbatim 环境的定制 5

4.1.1 注释 6

4.1.2 要忽略的起始字符 6

4.1.3 格式定制 7

4.1.4 逐行格式定制 7

4.1.5 字体 8

4.1.6 边框的类型与特性 9

4.1.7 环境标签 11

4.1.8 行号 13

4.1.9 要打印的行的选择 15

4.1.10 空格与制表符 17

4.1.11 行距 18

4.1.12 前后垂直间距 18

4.1.13 用于插入命令的转义字符 19

4.1.14 边距 20

4.1.15 Overfull box 提示 21

4.1.16 分页 21

4.1.17 字符类（catcode） 21

4.1.18 活动字符 22

4.1.19 引用标签 22
4.2 不同种类的 verbatim 环境 22
4.2.1 Verbatim 环境 22
4.2.2 BVerbatim 环境 22
4.2.3 LVerbatim 环境 23
4.2.4 自定义环境 23

5 verbatim 文本与环境的保存和恢复 24

6 verbatim 文件的读写 27

7 自动漂亮打印 28

8 已知问题 28

9 结论 29

II 宏包 fancyvrb-ex 30

10 Example 环境 30

11 CenterExample 环境 31

12 SideBySideExample 环境 31

Part I

宏包 fancyvrb

1 简介

‘fancyvrb’ 是对 ‘fancybox’ 宏包中 *verbatim* 宏的扩展，见 [3] 第 11 节。与标准 LaTeX 的 verbatim 环境相比，它提供了六类扩展功能：

1. verbatim 命令可以在脚注中使用；
2. 多个 verbatim 命令得到了增强；
3. 提供了多种 verbatim 环境，并带有大量参数来改变内容的排版方式；还可以自定义新的 verbatim 环境；
4. 提供了保存和恢复 verbatim 文本与环境的方法；
5. 提供了以 verbatim 模式写入和读取文件的宏，并具备通常的灵活性；
6. 可以构建 示例 环境（同时展示结果和 verbatim 文本），其灵活性与普通 verbatim 相同。

本宏包的工作方式是从环境或文件中逐行扫描内容。这样就可以在按照相应 catcode 执行一行内容之前，先对该行做预处理——拒绝该行、去掉行首空格、给它编号等等。

2 脚注中的 verbatim 内容

在声明 \VerbatimFootnotes 宏之后（在导言区之后使用），就可以在脚注中使用 verbatim 命令和环境（LaTeX 原生或 ‘fancyvrb’ 提供的），这与标准 LaTeX 不同：

```
\VerbatimFootnotes
我们可以把 verbatim\footnote{\verb+_可以！_+} 文本放进脚注里。
```

我们可以把 verbatim[1] 文本放进脚注里。

[1] _可以！_

3 增强的 verbatim 命令

\DefineShortVerb 宏可以把某个特殊字符定义为包裹 verbatim 文本的缩写定界符，而 \UndefineShortVerb 宏则取消指定字符的特殊含义（LaTeX 原生的 shortvrb 宏包也提供相同功能）：

```
\DefineShortVerb{\|}
我们可以只用单个 |_分隔符_| 来写
\Verb+_verbatim_+ 内容
\UndefineShortVerb{\|}
\DefineShortVerb{\+}
而且我们可以 +_换_+ 一个字符。
```

我们可以只用单个 _分隔符_ 来写_verbatim_ 内容 而且我们可以 _换_ 一个字符。

为了更加灵活，我们还可以在 verbatim 文本中指定转义字符（使用 \Verb 宏或已定义的 shortverb 字符），以便进行排版等操作；具体做法是使用 commandchars 参数，见第 4.1.13 节。

4 Verbatim 环境

fancyvrb 提供了多个 verbatim 环境，每个都有大量参数可供定制。下面的示例使用 Verbatim 环境，它等价于标准 verbatim。参数可以用 \fvset 宏全局设置，也可以在环境开始后的可选参数中设置[2, 3]。

```
\begin{Verbatim}
  第一行 verbatim。
  第二行 verbatim。
\end{Verbatim}
```

```
  第一行 verbatim。
  第二行 verbatim。
```

4.1 verbatim 环境的定制

verbatim 环境的外观可以通过多种方式改变；下面列出可供设置的键。

[2] 为便于阅读，本文中 verbatim 的每一行一般都缩进两个空格。

[3] 该机制使用标准 LaTeX 图形发行版中的 keyval 宏包，由 David Carlisle 编写。

4.1.1 注释

commentchar (character) ：在 verbatim 代码中用于定义注释的字符；以此字符开头的行不会被排版（默认：空）。

```
    一行注释
    verbatim 行。
```

```
\begin{Verbatim}[commentchar=!]
  一行注释
  verbatim 行。
  ! 这一行注释你将看不到
\end{Verbatim}
```

注意：如果注释字符不是一行中第一个非空白字符，会有特殊效果——这是因为该字符实际上被当作 TEX 的注释字符处理，也就是说它会同时吞掉换行符。因此在这种情况下，当前行会与下一行连起来；而且如果最后一行含有注释，它还会丢失，因为 'fancyvrb' 只有在找到一行结束符之后才会打印这一行，而在这种情况下永远找不到结束符……

```
  第一行。   第二行。
```

```
\begin{Verbatim}[commentchar=\%]
第一行。% 第一行
第二行。
第三行。% 第三行丢失……
\end{Verbatim}
```

4.1.2 要忽略的起始字符

gobble (integer) ：每行开头要忽略的字符数（最多 9 个），在环境本身有缩进时尤其有用（默认：空 — 不忽略任何字符）。

```
1   verbatim 行。
```

```
1   verbatim 行。
```

```
1 atim 行。
```

```
1  \begin{Verbatim}
2    verbatim 行。
3  \end{Verbatim}
4
5  \begin{Verbatim}[gobble=2]
6    verbatim 行。
7  \end{Verbatim}
8
9  \begin{Verbatim}[gobble=8]
10   verbatim 行。
11 \end{Verbatim}
```

4.1.3 格式定制

formatcom (command) : 排版 verbatim 文本之前要执行的命令（默认：空）。

formatcom* (command) : 在已有定义之上追加定义。

```
1 第一行 verbatim。
2 第二行 verbatim。
```

```
1 \begin{Verbatim}[formatcom=\color{red}]
2 第一行 verbatim。
3 第二行 verbatim。
4 \end{Verbatim}
```

```
1 第一行 verbatim。
2 第二行 verbatim。
```

```
1 \fvset{formatcom=\color{red}}
2 \begin{Verbatim}[formatcom*=\itshape]
3 第一行 verbatim。
4 第二行 verbatim。
5 \end{Verbatim}
```

4.1.4 逐行格式定制

\FancyVerbFormatLine 宏定义了每一行的排版方式。它的默认值是 \def\FancyVerbFormatLine#1{#1}，但我们可以按需要重定义（FancyVerbLine 是行计数器的名字）：

```
⇒ 第一行 verbatim。
⇒ 第二行 verbatim。
⇒ 第三行 verbatim。
```

```
  \renewcommand{\FancyVerbFormatLine}[1]{%
  \makebox[0cm][l]{$\Rightarrow$}#1}
\begin{Verbatim}
  第一行 verbatim。
  第二行 verbatim。
  第三行 verbatim。
\end{Verbatim}
```

```
  第一行 VERBATIM。
  第二行 verbatim。
  第三行 VERBATIM。
```

```
\renewcommand{\FancyVerbFormatLine}[1]{%
  \ifodd\value{FancyVerbLine}%
    \MakeUppercase{#1}\else#1\fi}
\begin{Verbatim}
  第一行 verbatim。
  第二行 verbatim。
  第三行 verbatim。
\end{Verbatim}
```

4.1.5 字体

fontfamily (family name)：使用的字体族。预定义有 `tt`、`courier` 和 `helvetica`（默认：*tt*）。

我们假定 PostScript 字体可用

```
verbatim 行。
```

```
\begin{Verbatim}[fontfamily=helvetica]
  verbatim 行。
\end{Verbatim}
```

fontsize (font size)：使用的字号（默认：*auto*—与当前字体相同）。如果你还使用了 relsize 宏包，可以要求字号按当前字号成比例变化（例如 `fontsize=\relsize{-2}`）。

我们假定 PostScript 字体可用

```
verbatim 行。
```

```
verbatim 行。
```

```
\begin{Verbatim}[fontsize=\small]
  verbatim 行。
\end{Verbatim}

\begin{Verbatim}[fontfamily=courier,
                 fontsize=\large]
  verbatim 行。
\end{Verbatim}
```

fontshape (font shape) : 使用的字体形状（默认：*auto*—与当前字体相同）。

我们假定 PostScript 字体可用

```
verbatim 行。
```

```
\begin{Verbatim}[fontfamily=courier,
                 fontshape=it]
  verbatim 行。
\end{Verbatim}
```

fontseries (series name) : 使用的 LaTeX 字体" 序列"（默认：*auto*—与当前字体相同）。

我们假定 PostScript 字体可用

```
verbatim 行。
```

```
\begin{Verbatim}[fontfamily=courier,
                 fontseries=b]
  verbatim 行。
\end{Verbatim}
```

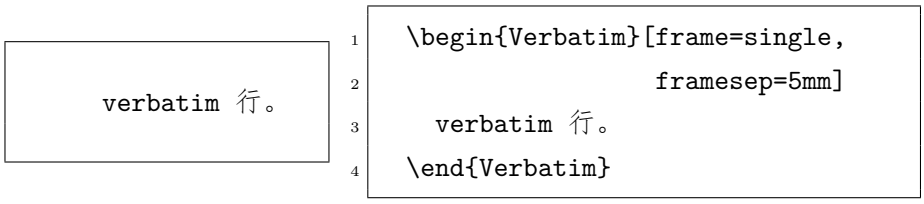
4.1.6 边框的类型与特性

framerule (dimension) : 边框线的宽度（默认：指定边框时为 *0.4pt*）。

```
verbatim 行。
```

```
\begin{Verbatim}[frame=single,
                 framerule=1mm]
  verbatim 行。
\end{Verbatim}
```

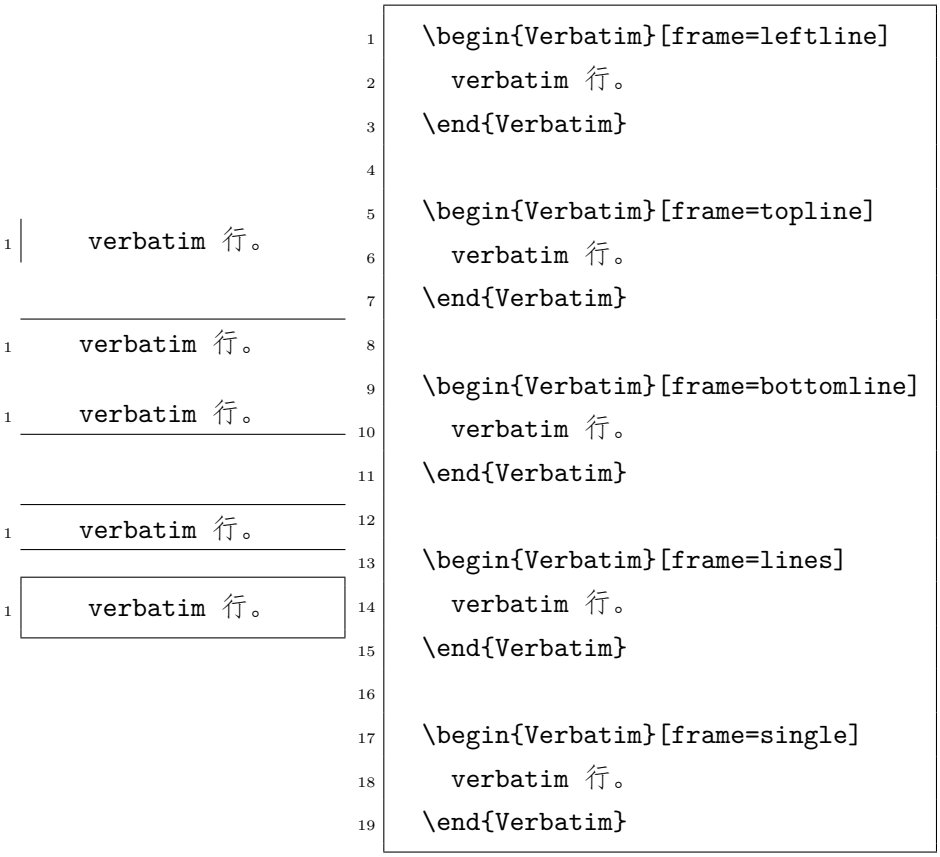
framesep (dimension) ：边框与文本之间的间距宽度（默认：*\fboxsep*）。



```
1 verbatim 行。
```

```
\begin{Verbatim}[frame=single,
                 framesep=5mm]
  verbatim 行。
\end{Verbatim}
```

frame (none|leftline|topline|bottomline|lines|single) ： verbatim 环境周围边框的类型（默认：*none*—无边框）。在 leftline 和 single 模式下，左边竖线与文本之间会按 LaTeX 的\fboxsep 长度留出一段间距。



```
1 verbatim 行。

1 verbatim 行。

1 verbatim 行。

1 verbatim 行。

1 verbatim 行。
```

```
\begin{Verbatim}[frame=leftline]
  verbatim 行。
\end{Verbatim}

\begin{Verbatim}[frame=topline]
  verbatim 行。
\end{Verbatim}

\begin{Verbatim}[frame=bottomline]
  verbatim 行。
\end{Verbatim}

\begin{Verbatim}[frame=lines]
  verbatim 行。
\end{Verbatim}

\begin{Verbatim}[frame=single]
  verbatim 行。
\end{Verbatim}
```

rulecolor (color command) ：边框线的颜色，用标准 LaTeX 方式指定（默认：*black*）。

```
verbatim 行。
```

```
\begin{Verbatim}[frame=single,
                 rulecolor=\color{red}]
  verbatim 行。
\end{Verbatim}
```

fillcolor (color command)：填充边框与文本之间空隙所用的颜色（其厚度由 framesep 给出）（默认：*none*—不填充颜色）。

```
verbatim 行。
```

```
\begin{Verbatim}[frame=single,
       framerule=1mm,framesep=3mm,
       rulecolor=\color{red},
       fillcolor=\color{yellow}]
  verbatim 行。
\end{Verbatim}
```

4.1.7 环境标签

label ({[string]string})：打印在环境顶边、底边或两边框线上的标签，用于描述环境内容（默认：空—无标签）。如果标签包含逗号或等号等特殊字符，必须把它放进一组花括号里。若只给一个字符串，则顶边和底边（若两者都打印）都用它；若还给了一个可选的首标签，则首标签用于顶边，第二个用于底边。还要注意：若 frame 参数取 topline、bottomline、lines、single 以外的值，标签将不会被打印。

注意在页顶处可能出现的问题……

我的文本
```
1 第一行 verbatim。
2 第二行 verbatim。
```

代码
```
1 第一行 verbatim。
2 第二行 verbatim。
```

```
1  \fvset{gobble=2}
2  \begin{Verbatim}[frame=single,
3                   label=我的文本]
4    第一行 verbatim。
5    第二行 verbatim。
6  \end{Verbatim}
7
8  \begin{Verbatim}[frame=topline,
9      framesep=4mm,
10     label=\fbox{\Large\emph{代码}}]
11   第一行 verbatim。
12   第二行 verbatim。
13 \end{Verbatim}
```

labelposition (none|topline|bottomline|all)：在定义了标签时，标签打印的位置；它必须与所选边框类型匹配（默认：标签为空时为 *none*；定义了一个标签时为 *topline*；定义了两个标签时为 *all*）。当然，某些不兼容的选项（例如 frame=topline,labelposition=bottomline）不会打印标签。

注意在页顶处可能出现的问题……

文本
```
1 第一行 verbatim。
2 第二行 verbatim。
```
文本

文本
```
1 第一行 verbatim。
2 第二行 verbatim。
```

```
1  \fvset{gobble=2}
2  \begin{Verbatim}[frame=single,
3         framesep=2mm,
4         label=文本,labelposition=all]
5    第一行 verbatim。
6    第二行 verbatim。
7  \end{Verbatim}
8
9  \begin{Verbatim}[frame=lines,
10        label=文本,labelposition=topline]
11   第一行 verbatim。
12   第二行 verbatim。
13 \end{Verbatim}
```

包含的代码

```
1 第一行 verbatim。
2 第二行 verbatim。
```

```
\begin{Verbatim}[frame=bottomline,
       framesep=3mm,
       label=\textit{包含的代码},
       labelposition=bottomline]
  第一行 verbatim。
  第二行 verbatim。
\end{Verbatim}
```

代码开头 — 代码结尾

```
1 第一行 verbatim。
2 第二行 verbatim。
```

```
\begin{Verbatim}[frame=lines,
                 framesep=3mm,
 label={[代码开头] 代码结尾}]
  第一行 verbatim。
  第二行 verbatim。
\end{Verbatim}
```

4.1.8 行号

numbers (none|left|right) ：verbatim 行的编号方式（默认：*none*—不编号）。
若启用，编号排在 verbatim 环境外侧。

```
1 第一行 verbatim。
2 第二行 verbatim。
```

```
\begin{Verbatim}[gobble=2,numbers=left]
  第一行 verbatim。
  第二行 verbatim。
\end{Verbatim}
```

```
第一行 verbatim。
第二行 verbatim。
```

```
\begin{Verbatim}[gobble=2,
       numbers=right,numbersep=0pt]
  第一行 verbatim。
  第二行 verbatim。
\end{Verbatim}
```

numbersep (dimension) ：编号与 verbatim 行之间的间距（默认：*12pt*）。

```
1 第一行 verbatim。
2 第二行 verbatim。
```

```
1  \begin{Verbatim}[gobble=2,
2        numbers=left,numbersep=2pt]
3    第一行 verbatim。
4    第二行 verbatim。
5  \end{Verbatim}
```

firstnumber (auto|last|integer) ：首行的编号（默认：*auto*—从 1 开始编号）。last 表示从上一个 verbatim 环境继续编号；若给出一个整数，则从该整数开始编号。

```
1   verbatim 行。
2   verbatim 行。
100 verbatim 行。
```

```
1  \fvset{gobble=2,
2        numbers=left,numbersep=3pt}
3  \begin{Verbatim}
4    verbatim 行。
5  \end{Verbatim}
6
7  \begin{Verbatim}[firstnumber=last]
8    verbatim 行。
9  \end{Verbatim}
10
11 \begin{Verbatim}[firstnumber=100]
12   verbatim 行。
13 \end{Verbatim}
```

stepnumber (integer) ：每隔多少行打印一次行号（默认：*1*—每行都编号）。

```
2 第一行 verbatim。
2 第二行 verbatim。
  第三行 verbatim。
```

```
1  \begin{Verbatim}[gobble=2,numbers=left,
2        numbersep=3pt,stepnumber=2]
3    第一行 verbatim。
4    第二行 verbatim。
5    第三行 verbatim。
6  \end{Verbatim}
```

\theFancyVerbLine 宏定义了行号的排版样式，所用的计数器是 FancyVerbLine：

```
\renewcommand{\theFancyVerbLine}{%
  \textcolor{red}{\small
    8.\alph{FancyVerbLine}}}
\begin{Verbatim}[gobble=2,
      numbers=left,numbersep=2pt]
  第一行 verbatim。
  第二行 verbatim。
  第三行 verbatim。
\end{Verbatim}
```

```
8.a 第一行 verbatim。
8.b 第二行 verbatim。
8.c 第三行 verbatim。
```

numberblanklines (boolean) ：是否给空行（真正空或只含空白字符的行）编号（默认：*true*—所有行都编号）。

```
\begin{Verbatim}[gobble=2,numbers=left,
      numbersep=3pt,
      numberblanklines=false]
  第一行 verbatim。


  第二行 verbatim。
\end{Verbatim}
```

```
1 第一行 verbatim。

2 第二行 verbatim。
```

4.1.9 要打印的行的选择

firstline (integer) ：要打印的第一行（默认：空—从第一行开始打印所有行）。

lastline (integer) ：要打印的最后一行（默认：空—一直打印到最后一行）。

```
\begin{Verbatim}[gobble=2,firstline,lastline,
      numbers=left,numbersep=2pt]
  第一行 verbatim。
  第二行 verbatim。
  第三行 verbatim。
\end{Verbatim}
```

```
1 第一行 verbatim。
2 第二行 verbatim。
3 第三行 verbatim。
```

```
  第二行 verbatim。
  第三行 verbatim。
```

```
\begin{Verbatim}[gobble=2,firstline=2,
      numbers=left,numbersep=2pt]
  第一行 verbatim。
  第二行 verbatim。
  第三行 verbatim。
\end{Verbatim}
```

```
  第一行 verbatim。
```

```
\begin{Verbatim}[gobble=2,lastline=1,
      numbers=left,numbersep=2pt]
  第一行 verbatim。
  第二行 verbatim。
  第三行 verbatim。
\end{Verbatim}
```

你也可以不指定 firstline 起止行号，而是定义一个起始字符串；区间的起点就是与该字符串完全相等的第一行。（比较是在从行首忽略任何字符之前进行的。）停止字符串类似。可以混用行号和字符串，例如从 firstline 开始、到某个停止字符串结束。指定字符串稍微有点笨拙：一开始必须用 \newcommand* 定义这两个字符串，如下所示：

```
第二行 verbatim。
```

```
\newcommand*\FancyVerbStartString{FROM}
\newcommand*\FancyVerbStopString{TO}
\begin{Verbatim}
  第一行 verbatim。
FROM
  第二行 verbatim。
TO
  第三行 verbatim。
\end{Verbatim}
```

要重新定义这两个字符串，必须使用 \renewcommand*。

若设置 lastline= 或 lastline=0，则什么都不打印。

```
foo
\begin{Verbatim}[frame=none,lastline=]
  第一行 verbatim。
  第二行 verbatim。
  第三行 verbatim。
\end{Verbatim}
bar
```

foo

bar

4.1.10 空格与制表符

showspaces (boolean) ： 为每个空格打印一个特殊符号（默认：*false*—不显示空格）。

```
verbatim␣行␣␣␣␣␣␣␣␣.
```

```
\begin{Verbatim}[showspaces,showtabs]
verbatim 行         .
\end{Verbatim}
```

实际上，所有 verbatim 环境、`\VerbatimInput` 和 `\Verb` 都有一个`*` 变体，它会把 `showspaces` 设为 true：

```
verbatim␣行␣␣␣␣␣␣␣␣.
```

```
\begin{Verbatim*}
verbatim 行         .
\end{Verbatim*}
```

```
\begin{filecontents}
  [noheader,force]{foo.txt}
foo        bar  baz
\end{filecontents}
\Verb|带制表符的 verbatim         .|
\Verb*|带制表符的 verbatim         .|

\VerbatimInput*{foo.txt}
```

带制表符的 verbatim
带制表符的␣verbatim␣␣␣␣␣␣␣␣

```
foo␣␣␣␣␣␣␣␣bar␣␣baz
```

还有一些参数用于决定制表符的解释方式（实际上使用制表符是相当老式的编码风格）：

showtabs (boolean) ： 显式显示制表符（默认：*false*—不显示制表符）。

obeytabs (boolean)：按制表符定位字符（默认：*false*—制表符直接加到当前位置）。

tabsize (integer)：一个制表符相当于多少个空格（默认：*8*）。

注意：LaTeX 内核提供的环境/命令行为与此不同。在 LaTeX 中制表符打印为可见空格，但用 `fancyvrb` 时则当作普通空格，或在 `showtabs` 下显示为可见制表符符号。

4.1.11 行距

baselinestretch (auto|dimension)：赋给常用 `baselinestretch` LaTeX 参数的值（默认：*auto*—取 verbatim 命令之前的当前值）。

```
第一行 verbatim。
第二行 verbatim。
```

```
\begin{Verbatim}[baselinestretch=2]
  第一行 verbatim。
  第二行 verbatim。
\end{Verbatim}
```

4.1.12 前后垂直间距

vspace (dimension)：赋给常用垂直列表间距的值（默认：*\topsep*—取 verbatim 命令之前的当前值）。

Verbatim 前面一行

```
foo
第一行 verbatim。
第二行 verbatim。
```

Verbatim 后面一行

```
Verbatim 前面一行
\begin{Verbatim}[frame=lines,label=foo]
  第一行 verbatim。
  第二行 verbatim。
\end{Verbatim}
Verbatim 后面一行
```

Verbatim 前面一行

foo

```
1 第一行 verbatim。
2 第二行 verbatim。
```

Verbatim 后面一行

```
1 Verbatim 前面一行
2 \begin{Verbatim}[frame=lines,label=foo,
3                  vspace=0pt]
4   第一行 verbatim。
5   第二行 verbatim。
6 \end{Verbatim}
7 Verbatim 后面一行
```

4.1.13 用于插入命令的转义字符

commandchars (three characters) ： 用于指定哪个字符表示宏开始、以及哪两个字符标记一组的开始和结束；从而可以在 verbatim 代码中引入转义序列。当然，最好选择 verbatim 文本中用不到的特殊字符！（默认：空）。

```
1 这是一条注释
2 第一行 verbatim。
3 第二行 verbatim。
4 第三行 verbatim。
```

```
1 \textbf{verbatim 行}。
```

```
1  \begin{Verbatim}[commandchars=\\\{\}]
2    \textit{这是一条注释}
3    第一行 verbatim。
4    \fbox{第二}行 verbatim。
5    \textcolor{red}{第三}行 verbatim。
6  \end{Verbatim}
7
8  \begin{Verbatim}[commandchars=+\[\]]
9    +textit[\textbf{verbatim 行}]。
10 \end{Verbatim}
```

需要对] 做转义，以避免末尾出现不允许的]]。或者你也可以写成 commandchars={+[]} 或 commandchars=+{[}{]}。对所有活动字符都需要转义。注意不要使用 fancyvrb 自身代码中已用到的字符，例如 <>。

用这种方式，还可以放置标签，以便日后引用 verbatim 环境中的某些行：

```
1  第一行 verbatim。
2  第二行。
3  第三行 verbatim。
```

如前面所示，第 2 行是……

```
1  \begin{Verbatim}[commandchars=\\\{\},
2        numbers=left,numbersep=2pt]
3    第一行 verbatim。
4    第二行。\label{vrb:Important}
5    第三行 verbatim。
6  \end{Verbatim}
7
8    如前面所示，
9  第~\ref{vrb:Important} 行是……
```

4.1.14 边距

xleftmargin (dimension) ： 每行开头要增加的缩进（默认：*0pt*—左边距为零）。

```
1  verbatim 行。
```

```
1  \begin{Verbatim}[frame=single,
2                   xleftmargin=5mm]
3    verbatim 行。
4  \end{Verbatim}
```

xrightmargin (dimension) ： 每行末尾要增加的右边距（默认：*0pt*—右边距为零）。

```
1  verbatim 行。
```

```
1  \begin{Verbatim}[frame=single,
2                   xrightmargin=1cm]
3    verbatim 行。
4  \end{Verbatim}
```

resetmargins (boolean) ： 重置左边距；当我们处于其他有缩进的环境内部时很有用（默认：*false*—不重置边距）。

- 第一个条目

```
verbatim 行。
```

- 第二个条目

```
verbatim 行。
```

```
\begin{itemize}
  \item 第一个条目
  \begin{Verbatim}[frame=single]
verbatim 行。
  \end{Verbatim}
  \item 第二个条目
  \begin{Verbatim}[frame=single,
                   resetmargins=true]
verbatim 行。
  \end{Verbatim}
\end{itemize}
```

4.1.15 Overfull box 提示

hfuzz (dimension)：赋给 TEX \hfuzz 尺寸的值，用于排版文本。这可以用来避免看到一些不重要的 *Overfull box*（盒子溢出）提示（默认：*2pt*）。

4.1.16 分页

samepage (boolean)：在非常特殊的情况下，我们可能希望保证一个 verbatim 环境不被分页拆开，即使它在当前页放不下。为避免分页，可以把 samepage 参数设为 *true*（默认：*false*）。

4.1.17 字符类（catcode）

codes (macro)：指定 *catcode*（字符类）改动（默认：空）。

codes* (macro)：在已有定义之上追加。

例如，这使我们可以在 verbatim 文本中插入排版好的数学公式：

```
x=1/sqrt(z**2) ! 1/√(z²)
```

```
\begin{Verbatim}[commandchars=\\\{\},
         codes={\catcode`$=3\catcode`^=7}]
    x=1/sqrt(z**2) ! $\frac{1}{\sqrt{z^2}}$
\end{Verbatim}
```

用 codes* 可以在已有的 codes 定义之上追加代码：

```
x=1/sqrt(z**2) ! 1/√(z^2)
```

```
\fvset{codes={\catcode`$=3\catcode`^=7}}
\begin{Verbatim}[commandchars=\\\{\},
  % 在 codes 之上追加
  codes*={\color{blue}}]
x=1/sqrt(z**2) ! $\frac{1}{\sqrt{z^2}}$
\end{Verbatim}
```

4.1.18 活动字符

defineactive (macro)：定义 *活动* 字符的效果（默认：空）。

defineactive* (macro)：在已有定义之上追加。

这让我们可以玩一些巧妙的花样；见第 27 页第 6 节的示例。

4.1.19 引用标签

reflabel (<label>)：供 \pageref 使用的标签。

```
    第一行 verbatim。
    第二行 verbatim。
```

见第 22 页上的 verbatim。

```
\begin{Verbatim}[reflabel=verb0]
  第一行 verbatim。
  第二行 verbatim。
\end{Verbatim}
见第~\pageref{verb0} 页上的 verbatim。
```

4.2 不同种类的 verbatim 环境

4.2.1 Verbatim 环境

这就是我们一直用到现在的”普通” verbatim 环境。

4.2.2 BVerbatim 环境

这个环境把 verbatim 内容放进一个 TEX 盒子里。有些参数在该环境内不起作用（尤其是边框相关的参数），但新增了两个可用参数：

boxwidth (auto|dimension)：所用盒子的尺寸（默认：*auto*—取最长一行的宽度）。

baseline (b|c|t) ：基线位置（在盒子的 baseline、center 或 top）（默认：*b*）。

第一行
第二行　第一行
　　第二行

```
\fvset{gobble=2}
\begin{BVerbatim}
  第一行
  第二行
\end{BVerbatim}
\begin{BVerbatim}[baseline=c]
  第一行
  第二行
\end{BVerbatim}
```

第一行
第二行　第一行
　　第二行

```
\begin{BVerbatim}[boxwidth=2cm]
  第一行
  第二行
\end{BVerbatim}
\begin{BVerbatim}[boxwidth=2cm,
                  baseline=t]
  第一行
  第二行
\end{BVerbatim}
```

4.2.3 LVerbatim 环境

这个环境把 verbatim 内容放入 LaTeX 的"LR" 模式（即所谓从左到右模式，实际上与 TeX 自己所说的 受限水平模式是同一回事）。

4.2.4 自定义环境

自定义环境非常容易。你可以用 \RecustomVerbatimEnvironment 宏重定义已有环境，或用 \DefineVerbatimEnvironment 宏创建自己的环境[4]。

[4]对于 verbatim 命令，还有 \CustomVerbatimCommand 和\RecustomVerbatimCommand 宏；例如：\RecustomVerbatimCommand{\VerbatimInput}{VerbatimInput}{frame=lines}

无论哪种方式，都要指定新环境的名字、它所基于的环境类型，以及一组初始选项值。这些选项仍可用常规的可选参数覆盖：

```
第一行 verbatim。
第二行 verbatim。
```

```
\RecustomVerbatimEnvironment
  {Verbatim}{Verbatim}
  {gobble=2,frame=single}
\begin{Verbatim}
  第一行 verbatim。
  第二行 verbatim。
\end{Verbatim}
```

```
第一行 verbatim。
第二行 verbatim。
```

```
\DefineVerbatimEnvironment%
  {MyVerbatim}{Verbatim}
  {gobble=2,numbers=left,numbersep=2mm,
   frame=lines,framerule=0.8mm}
\begin{MyVerbatim}
  第一行 verbatim。
  第二行 verbatim。
\end{MyVerbatim}

\begin{MyVerbatim}[numbers=none,
                   framerule=1pt]
  第一行 verbatim。
  第二行 verbatim。
\end{MyVerbatim}
```

```
第一行 verbatim。
第二行 verbatim。
```

5 verbatim 文本与环境的保存和恢复

\SaveVerb 和 \UseVerb 宏让我们可以保存和恢复 verbatim 内容。\UseVerb 本身是健壮的：

我已保存 _verbatim_，以后想重复用多少次都行

```
\DefineShortVerb{\|}
\SaveVerb{Verb}|_verbatim_|
我已保存 \UseVerb{Verb}，
以后想重复用多少次都行
\subsection*{使用 \UseVerb{Verb}}
\UseVerb{Verb}。
```

使用 _verbatim_

verbatim。

这同时也为”在通常不允许 verbatim 的 LaTeX 命令中放入 verbatim 文本” 提供了解决方案：

```
\DefineShortVerb{\|}\SaveVerb{Verb}|_OK^| \marginpar{\UseVerb{Verb}}
```

_OK^

还有一个很有用的功能：可以在 description 列表中把 verbatim 文本作为条目文本（这在标准 LaTeX 中通常是不允许的），方法是使用 aftersave 参数：

aftersave (macro)：用于动态保存某些 verbatim 内容的宏（默认：空）。

```
\newcommand{\Vitem}{%
  \SaveVerb[aftersave={%
    \item[\UseVerb{Vitem}]}]{Vitem}}
\DefineShortVerb{\|}
\begin{description}
  \Vitem|\MyCommand|: 我的命令
\end{description}
```

\MyCommand：我的命令

同样，我们可以保存并恢复完整的 verbatim 环境（分别用 \UseVerbatim、\BUseVerbatim 和 \LUseVerbatim，对应普通模式、盒子模式和 LR 模式）：

```
    verbatim 行。
```

以及

```
    verbatim 行。
```

```
  \begin{SaveVerbatim}{VerbEnv}
  verbatim 行。
\end{SaveVerbatim}
\UseVerbatim{VerbEnv}
以及 \UseVerbatim{VerbEnv}
```

一行
二行 以及 一行
二行。

```
一行
二行
```

以及

```
一行
二行
```

```
    \begin{SaveVerbatim}[gobble=5]{VerbEnv}
    第一行
    第二行
  \end{SaveVerbatim}

  \fbox{\BUseVerbatim{VerbEnv}}
  以及 \BUseVerbatim{VerbEnv}。

  \LUseVerbatim{VerbEnv} 以及
  \LUseVerbatim{VerbEnv}
```

```
这样是有效的。
我直接用 Verbatim。
```

```
不过,如果我用 SaveVerbatim
定义一个可复用的 Verbatim,
行号是生效的。
```

```
\begin{SaveVerbatim}{FOO}
不过，如果我用 SaveVerbatim
定义一个可复用的 Verbatim，
行号是生效的。
\end{SaveVerbatim}

\begin{Verbatim}[numbers=left]
这样是有效的。
我直接用 Verbatim。
\end{Verbatim}

\UseVerbatim[numbers=left]{FOO}
```

6 verbatim 文件的读写

\VerbatimInput 命令（也有 \BVerbatimInput 和\LVerbatimInput 变体）允许以 verbatim 排版方式包含一个文件的内容。当然，前面介绍过的各种定制参数仍然可以使用：

```
! A "hello" program

program hello
  print *,"Hello world"
end program hello
```

```
! A "hello" program

program hello
  print *,"Hello world"
end program hello
```

```
program hello
  print *,"Hello world"
end program hello
```

```
! A "hello" program

program hello
  print *,"Hello world"
end program hello
```

```
  \fvset{fontsize=\small}
\VerbatimInput{hello.f90}

\fvset{frame=single,numbers=left,
       numbersep=3pt}
\VerbatimInput{hello.f90}

\VerbatimInput[firstline=3,
    rulecolor=\color{green}]
  {hello.f90}

\VerbatimInput[frame=lines,
    fontshape=sl,fontsize=\footnotesize]
  {hello.f90}
```

我们可以利用 defineactive 参数，用不同样式排版程序文本中的注释行：

```
! A "hello" program

program hello
  print *,"Hello world"
end program hello
```

```
  \def\ExclamationPoint{\char33}
\catcode`!=\active
\VerbatimInput%
  [defineactive=%
    \def!{\color{cyan}\itshape
      \ExclamationPoint}]
  {hello.f90}
```

要注意：如果文件内容在一页内排不下，它会按需要自动跨页拆分（除非 `samepage` 参数被设为 `true`）。

还有一个 `VerbatimOut` 环境，可以用同样的方式把 verbatim 文本写入输出文件：

```
1  我写这一行。
2  还有这一行。
```

```
1  \begin{VerbatimOut}{file.txt}
2    我写这一行。
3    还有这一行。
4  \end{VerbatimOut}
5
6  \VerbatimInput[frame=single,
7    numbers=left,numbersep=6pt]{file.txt}
```

7 自动漂亮打印

显然，自动 *漂亮打印*（pretty printing）不在本宏包的范围内。尽管如此，它对于以 verbatim 方式包含程序代码文件或代码片段特别有意义。在 LaTeX 世界里（暂不考虑 *文学编程* 方式），针对某些特殊语言有相应的软件，例如 Norbert Kiesel 编写的 C++2LaTeX 宏包；但主要有两个通用工具，二者采用完全不同的模式（一个是用 C 编写的外部预处理器，另一个是基于 TeX 的方案）：一个是 LGrind [2] 系统（目前由 Michael Piefel 维护），另一个是 Carsten Heinz 编写的 listings [1] 宏包。

‘`fancyvrb`’ 与 listings 宏包的未来版本计划协同工作，这将给两个宏包的现有用户都带来很大好处，并使 ‘`fancyvrb`’ 用户能够获得程序代码的自动漂亮打印。

8 已知问题

- Vladimir Volovich `<vvv@vvv.vsu.ru>` 报告说：用 T1 编码可用的特殊字符 `\th`，无法作为 verbatim 内容被 ‘`fancyvrb`’ 包含。其他特殊字符也可能有同样问题。

9 结论

还有一些其他功能我们在此没有介绍。特别要注意：你可以定义一个定制文件（`fancyvrb.cfg`），它会在宏包末尾被加载，用于存放你自定义命令和环境的定义，以及重定义已有环境的属性。

Part II

宏包 fancyvrb-ex

这个宏包定义了一些示例环境，可以把输入代码和输出并排或上下排列。本文档中所有 fancyvrb 的示例都用到了它们。

10 Example 环境

```
\begin{Example}
  第一行 verbatim。
  第二行 verbatim。
  第三行 verbatim。
\end{Example}
```

```
  第一行 verbatim。
  第二行 verbatim。
  第三行 verbatim。
```

第一行 verbatim。第二行 verbatim。第三行 verbatim。

```
\begin{Example}[frame=lines,framerule=1mm,
        numbers=left]
  第一行 verbatim。
  第二行 verbatim。
  第三行 verbatim。
\end{Example}
```

```
  第一行 verbatim。
  第二行 verbatim。
  第三行 verbatim。
```

第一行 verbatim。第二行 verbatim。第三行 verbatim。

11 CenterExample 环境

```
\begin{CenterExample}[frame=single,
    numbers=right]
  第一行 verbatim。
  第二行 verbatim。
  第三行 verbatim。
\end{CenterExample}
```

```
  第一行 verbatim。
  第二行 verbatim。
  第三行 verbatim。
```

第一行 verbatim。第二行 verbatim。第三行 verbatim。

12 SideBySideExample 环境

```
\begin{SideBySideExample}[xrightmargin=5cm,
   frame=lines, numbers=left]
  第一行 verbatim。
  第二行 verbatim。
  第三行 verbatim。
\end{SideBySideExample}
```

第一行 verbatim。第二行 verbatim。第三行 verbatim。

```
  第一行 verbatim。
  第二行 verbatim。
  第三行 verbatim。
```

参考文献

[1] Carsten Heinz and Brooks Moses. *The listings package*. CTAN Comprehensive TeX Archive Network. 2020. URL: https://www.ctan.org/pkg/listings.

[2] Michael Piefel. *The LGrind system*. CTAN Comprehensive TeX Archive Network. URL: https://www.ctan.org/pkg/lgrind.

[3] Timothy Van Zandt, Hans Hagen, and Heiko Oberdiek. *The fancybox package*. CTAN Comprehensive TeX Archive Network. 2001. URL: https://www.ctan.org/pkg/fancybox.

[4] Timothy Van Zandt et al. *The fancyvrb package: Fancy Verbatims in LaTeX*. CTAN Comprehensive TeX Archive Network. 2022. URL: https://www.ctan.org/pkg/fancyvrb.